

1.2 Second Example

Our second example is of little more practical use than the first, but it introduces a few more programming language elements:

```
#include <stdio.h>

/* print a few numbers, to illustrate a simple loop */

main()
{
    int i;

    for(i = 0; i < 10; i = i + 1)
        printf("i is %d\n", i);

    return 0;
}
```

As before, the line `#include <stdio.h>` is boilerplate which is necessary since we're calling the `printf` function, and `main()` and the pair of braces `{}` indicate and delineate the function named `main` we're (again) writing.

The first new line is the line

```
/* print a few numbers, to illustrate a simple loop */
```

which is a *comment*. Anything between the characters `/*` and `*/` is ignored by the compiler, but may be useful to a person trying to read and understand the program. You can add comments anywhere you want to in the program, to document what the program is, what it does, who wrote it, how it works, what the various functions are for and how *they* work, what the various variables are for, etc.

The second new line, down within the function `main`, is

```
int i;
```

which *declares* that our function will use a variable named `i`. The variable's type is `int`, which is a plain integer.

Next, we set up a *loop*:

```
for(i = 0; i < 10; i = i + 1)
```

The keyword `for` indicates that we are setting up a ``for loop." A `for` loop is controlled by three expressions, enclosed in parentheses and separated by semicolons. These expressions say that, in this case, the loop starts by setting `i` to 0, that it continues as long as `i` is less than 10, and that after each iteration of the loop, `i` should be incremented by 1 (that is, have 1 added to its value).

Finally, we have a call to the `printf` function, as before, but with several differences. First, the call to `printf` is within the *body* of the `for` loop. This means that control flow does not pass once through the `printf` call, but instead that the call is performed as many times as are dictated by the `for` loop. In this case, `printf` will be called several times: once when `i` is 0, once when `i` is 1, once when `i` is 2, and so on until `i` is 9, for a total of 10 times.

A second difference in the `printf` call is that the string to be printed, `"i is %d"`, contains a percent

sign. Whenever `printf` sees a percent sign, it indicates that `printf` is not supposed to print the exact text of the string, but is instead supposed to read another one of its arguments to decide what to print. The letter after the percent sign tells it what type of argument to expect and how to print it. In this case, the letter `d` indicates that `printf` is to expect an `int`, and to print it in decimal. Finally, we see that `printf` is in fact being called with another argument, for a total of two, separated by commas. The second argument is the variable `i`, which is in fact an `int`, as required by `%d`. The effect of all of this is that each time it is called, `printf` will print a line containing the current value of the variable `i`:

```
i is 0
i is 1
i is 2
...
```

After several trips through the loop, `i` will eventually equal 9. After that trip through the loop, the third control expression `i = i + 1` will increment its value to 10. The condition `i < 10` is no longer true, so no more trips through the loop are taken. Instead, control flow jumps down to the statement following the `for` loop, which is the `return` statement. The `main` function returns, and the program is finished.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail](#) [feedback](#)